arXiv:2006.11316v1 [cs.CL] 19 Jun 2020

SqueezeBERT: What can computer vision teach NLP about efficient neural networks?

Forrest N. Iandola
forresti@berkeley.edu

Albert E. Shaw
ashaw596@gmail.com

Ravi Krishna
UC Berkeley EECS
ravi.krishna@berkeley.edu

Kurt W. Keutzer
UC Berkeley EECS
keutzer@berkeley.edu

Abstract

Humans read and write hundreds of billions of messages every day. Further, due to the availability of large datasets, large computing systems, and better neural network models, natural language processing (NLP) technology has made significant strides in understanding, proofreading, and organizing these messages. Thus, there is a significant opportunity to deploy NLP in myriad applications to help web users, social networks, and businesses. In particular, we consider smartphones and other mobile devices as crucial platforms for deploying NLP models at scale. However, today's highly-accurate NLP neural network models such as BERT and RoBERTa are extremely computationally expensive, with BERT-base taking 1.7 seconds to classify a text snippet on a Pixel 3 smartphone. In this work, we observe that methods such as grouped convolutions have yielded significant speedups for computer vision networks, but many of these techniques have not been adopted by NLP neural network designers. We demonstrate how to replace several operations in self-attention layers with grouped convolutions, and we use this technique in a novel network architecture called SqueezeBERT, which runs 4.3x faster than BERT-base on the Pixel 3 while achieving competitive accuracy on the GLUE test set. The SqueezeBERT code will be released.

1 Introduction and Motivation

The human race writes over 300 billion messages per day [1, 2, 3, 4]. Out of these, more than half of the world's emails are read on mobile devices, and nearly half of Facebook users exclusively access Facebook from a mobile device [5, 6]. Natural language processing (NLP) technology has the potential to aid these users and communities in several ways. When a person writes a message, NLP models can help with spelling and grammar checking as well as sentence completion. When content is added to a social network, NLP can facilitate content moderation before it appears in other users' news feeds. When a person consumes messages, NLP models can help classify messages into folders, composing news feeds, prioritizing messages, and identifying duplicate messages.

In recent years, the development and adoption of Attention Neural Networks has led to dramatic improvements in almost every area of NLP. In 2017, Vaswani *et al.* proposed the multi-head self-attention module, which demonstrated superior accuracy to recurrent neural networks on English-German machine language translation [7].[1] These modules have since been adopted by GPT [8] and BERT [9] for sentence classification, and by GPT-2 [10] and CTRL [11] for sentence completion and generation. Recent works such as ELECTRA [12] and RoBERTa [13] have shown that larger datasets and more sophisticated training regimes can further improve the accuracy of self-attention networks.

Considering the enormity of the textual data created by humans on mobile devices, a natural approach is to deploy NLP models themselves on the mobile devices themselves, embedding them

[1] Neural networks that use the self-attention modules of Vaswani *et al.* are sometimes called "Transformers," but in the interest of clarity we call them "self-attention networks."

in common apps that are used to read, write, and share text. Unfortunately, many of today's best state-of-the-art NLP models may are often rather computationally expensive, often making mobile deployment impractical. For example, we observe that running the BERT-base network on a Google Pixel 3 smartphone approximately 1.7 seconds to classify a single text data sample.[2] Much of the research on efficient self-attention networks for NLP has just emerged in the past year. However, starting with SqueezeNet [14], the mobile computer vision (CV) community has spent the last four years optimizing neural networks for mobile devices. Intuitively, it seems like there must be opportunities to apply the lessons learned from the rich literature of mobile CV research to accelerate mobile NLP. In the following we review what has already been applied and propose two additional techniques from CV that we will leverage for accelerating NLP.

1.1 What has CV research already taught NLP research about efficient networks?

In recent months, novel self-attention networks have been developed with the goal of achieving faster inference. At present, the MobileBERT network defines the state-of-the-art in low-latency text classification for mobile devices [15]. MobileBERT takes approximately 0.6 seconds to classify a text sequence on a Google Pixel 3 smartphone. And, on the GLUE benchmark, which consists of 9 natural language understanding (NLU) datasets [16], MobileBERT achieves higher accuracy than other efficient networks such as DistilBERT [17], PKD [18], and several others [19, 20, 21, 22]. To achieve this, MobileBERT introduced two concepts into their NLP self-attention network that are already in widespread use in CV neural networks:

1. **Bottleneck layers.** In ResNet [23], the 3x3 convolutions are computationally expensive, so a 1x1 "bottleneck" convolution is employed to reduce the number of channels input to each 3x3 convolution layer. Similarly, MobileBERT adopts bottleneck layers that reduce the number of channels before each self-attention layer, and this reduces the computational cost of the self-attention layers.
2. **High-information flow residual connections.** In BERT-base, the residual connections serve as links between the low-channel-count (768 channels) layers. The high-channel-count (3072 channels) layers in BERT-base do not have residual connections. However, the ResNet and Residual-SqueezeNet [14] CV networks connect the high-channel-count layers with residuals, which enables higher information flow through the network. Similar to these CV networks, MobileBERT adds residual connections between the high-channel-count layers.

1.2 What else can CV research teach NLP research about efficient networks?

We are encouraged by the progress that MobileBERT has made in leveraging ideas that are popular in the CV literature to accelerate NLP.

However, we are aware of two other ideas from CV, which weren't used in MobileBERT, and that could be applied to accelerate NLP:

1. **Convolutions.** Since the 1980s, computer vision neural nets have relied heavily on convolutional layers [24, 25]. Convolutions are quite flexible and well-optimized in software, and they can implement things as simple as a 1D fully-connected layer, or as complex as a 3D dilated layer that performs upsampling or downsampling.
2. **Grouped convolutions.** A popular technique in modern mobile-optimized neural networks is grouped convolutions (see Section 3). Proposed by Krizhevsky *et al.* in the 2012 winning submission to the ImageNet image classification challenge [26, 27, 28], grouped convolutions disappeared from the literature from some years, then re-emerged as a key technique circa 2016 [29, 30] and today are extensively used in efficient CV networks such as MobileNet [31], ShuffleNet [32], and EfficientNet [33]. While common in efficient CV literature, we are not aware of work applying grouped convolutions to NLP.

1.3 SqueezeBERT: Applying lessons learned from CV to NLP

In this work, we describe how to apply convolutions and particularly grouped convolutions in the design of a novel self-attention network for NLP, which we call SqueezeBERT. Empirically, we

[2] Note that BERT-base [9], RoBERTa-base [13], and ELECTRA-base [12] all use the same self-attention encoder architecture, and therefore these networks incur approximately the same latency on a smartphone.

Table 1: **How does BERT spend its time?** This is a breakdown of computation (in floating-point operations, or FLOPs) and latency (on a Google Pixel 3 smartphone) in BERT-base, reported to three significant digits. The FC layers account for more than 97% of the FLOPs and over 88% of the latency.

Stage	Module type	FLOPs	Latency
Embedding	Embedding	0.00%	0.26%
Encoder	FC in self-attention modules	24.3%	18.9%
Encoder	$softmax(\frac{QK^T}{\sqrt{d_k}})V$ in self-attention modules	2.70%	11.3%
Encoder	FC in feed-forward network layers	73.0%	69.4%
Final Classifier	FC layers in final classifier	0.00%	0.02%
Total		100%	100%

find that SqueezeBERT runs at lower latency on a smartphone than BERT-base, MobileBERT, and several other efficient NLP models, while maintaining competitive accuracy.

2 Implementing self-attention with convolutions

In this section, first we review the basic structure of self-attention networks, next we identify that their biggest computational bottleneck is in their position-wise fully-connected (PFC) layers, and then we show that these PFC layers are equivalent to a 1D convolution with a kernel size of 1.

2.1 Self-attention networks

In most BERT-derived networks there are typically 3 stages: the embedding, the encoder, and the classifier [9, 13, 12, 15, 19]. [3] The embedding converts preprocessed words (represented as integer-valued tokens) into learned feature-vectors of floating-point numbers; the encoder is comprised of a series of self-attention and other layers; and the classifier produces the network's final output. As we will see later in Table 1, the embedding and the classifier account for less than 1% of the runtime of a self-attention network, so we focus our discussion on the encoder.

We now describe the encoder that is used in BERT-base [9]. The encoder consists of a stack of blocks. Each block consists of a self-attention module followed by three position-wise fully-connected layers, known as feed-forward network (FFN) layers. Each self-attention module contains three seperate position-wise fully-connected (PFC) layers, which are used to generate the *query* (Q), *key* (K), and *value* (V) activation vectors for each position in the feature embedding. Each of these PFC layers in self-attention applies the same operation to each position in the feature embedding independently. While neural networks traditionally multiply weights by activations, a distinguishing factor of attention neural networks is that they multiply activations by other activations, which enables dynamic weighting of tensor elements to adjust based on the input data. Further, attention networks allow modeling of arbitrary dependencies irregardless of their distance in the input or output [7]. The self-attention module proposed by Vaswani *et al.* [7] (which is also used by GPT [8], BERT [9], RoBERTa [13], ELECTRA [12] and others) multiplies the Q, K, and V activations together using the equation $softmax(\frac{QK^T}{\sqrt{d_k}})V$, where d_k is the number of channels in one attention head.[4]

2.2 Benchmarking BERT for mobile inference

To identify the parts of BERT that are time-consuming to compute, we profile BERT on a smartphone. Specifically, we measure the neural network's latency using PyTorch [34] and TorchScript on a Google Pixel 3 smartphone, with an input sequence length of 128 and a batch size of 1. In Table 1, we show the breakdown of FLOPs and latency among the main components of the BERT network, and we observe that the $softmax(\frac{QK^T}{\sqrt{d_k}})V$ calculations in the self-attention modules account for

[3] Some self-attention networks such as [7, 8] also have "decoder" stage. The decoder typically uses a similar neural architecture as the encoder, but is auto-regressive.

[4] For example, in BERT-base, the self-attention module has 768 channels and 12 heads, so $d_k = \frac{768}{12} = 64$.

only 11.3 percent of the total latency. However, the PFC layers in the self-attention modules account for 18.9 percent, and the PFC layers in the feed-forward network modules account for 69.4 percent, and in total the PFC layers account for 88.3 percent of the latency. Given that the PFC layers account for the overwhelming majority of the latency, we now turn our focus to reducing the latency of the PFC layers.

2.3 Replacing the position-wise fully connected layers (PFC) with convolutions

To address this, we intend to replace the PFC layers with grouped convolutions, which have been shown to produce significant speedups in computer vision networks. As a first step in this direction, we now show that the fully-connected layers used throughout the BERT encoder are a special case of non-grouped 1D convolution. In the following, f denotes the input feature vector, and w denotes the weights. Given an input feature vector of dimensions (P, C_{in}) with P positions and C_{in} channels to generate an output of (P, C_{out}) features, the operation performed by the position-wise fully-connected layer can be defined as follows:

$$PositionwiseFullyConnected_{p,c_{out}}(f, w) = \sum_{i}^{C_{in}} w_{c_{out},i} * f_{p,i} \tag{1}$$

Then if we consider the definition of a 1D convolution with kernel size K with the same input and output dimensions

$$Convolution_{p,c_{out}}(f, w) = \sum_{i}^{C_{in}} \sum_{k}^{K} w_{c_{out},i,k} * f_{(p-\frac{K-1}{2}+k),i} \tag{2}$$

we observe that the position-wise fully-connected operation is equivalent to a convolution with a kernel size of $k = 1$. Thus, the PFC layers of Vaswani *et al.* [7], GPT, BERT, and similar self-attention networks can be implemented using convolutions without changing the networks' numerical properties or behavior.

3 Incorporating grouped convolutions into self-attention

Now that we have shown how to implement the expensive PFC layers in self-attention networks using convolutions, we can incorporate efficient grouped convolutions into a self-attention network. Grouped convolutions are defined as follows. Given an input feature vector of dimensions (P, C_{in}) with P positions and C_{in} channels and outputting a vector with dimensions (P, C_{out}), a 1d convolution with kernel size K and G groups can be defined as follows.

$$GroupedConvolution_{p,c_{out}}(f, w) = \sum_{i}^{\frac{C_{in}}{G}} \sum_{k}^{K} w_{c_{out},i,k} * f_{(p-\frac{K-1}{2}+k),(i+\lfloor\frac{(i)(G)}{C_{out}}\rfloor\frac{C_{in}}{G})} \tag{3}$$

This is equivalent to splitting the the input vector into G separate vectors of size $(P, \frac{C_{in}}{G})$ along the P dimension and running G separate convolutions with independent weights each computing vectors of size $(P, \frac{C_{out}}{G})$. The grouped convolution, however, requires only $\frac{1}{G}$ as many floating point operations (FLOPs) and $\frac{1}{G}$ as many weights as an ordinary convolution, not counting the small (and unchanged) amount of operations needed for the channel-wise bias term that is often included in convolutional layers.[5]

3.1 SqueezeBERT

Now, we describe our proposed neural architecture called SqueezeBERT, which uses grouped convolutions. SqueezeBERT is much like BERT-base, but with PFC layers implemented as convolutions, and grouped convolutions for many of the layers. Recall from Section 2 that each block in the BERT-base encoder has a self-attention module with 3 PFC layers, plus 3 more PFC layers called feed-forward network layers (FFN_1, FFN_2, and FFN_3). The FFN layers have the following dimensions: FFN_1 has $C_{in} = C_{out} = 768$, FFN_2 has $C_{in} = 768$ and $C_{out} = 3072$, and FFN_3 has

[5]Note that the grouped convolution with $G = 1$ is identical to an ordinary convolution.

$C_{in} = 3072$ and $C_{out} = 768$. In all PFC layers of the self-attention modules, and in the FFN_2 and FFN_3 layers, we use grouped convolutions with $G = 4$. To allow for mixing across channels of different groups, we use $G = 1$ in the less-expensive FFN_1 layers. Note that in BERT-base, FFN_2 and FFN_3 each have 4 times more arithmetic operations than FFN_1. However, when we use $G = 4$ in FFN_2 and FFN_3, now all FFN layers have the same number of arithmetic operations.

Finally, the embedding size (768), the number of blocks in the encoder (12), the number of heads per self-attention module (12), the tokenizer (WordPiece [35, 36]), and other aspects of SqueezeBERT are adopted from BERT-base. Aside from the convolution-based implementation and the adoption of grouped convolutions, the SqueezeBERT architecture is identical to BERT-base.

4 Experimental Methodology

4.1 Datasets

Pretraining Data. For pretraining, we use a combination of Wikipedia and BooksCorpus [37], setting aside 3% of the combined dataset as a test set. Following the ALBERT paper, we use Masked Language Modeling (MLM) and Sentence Order Prediction (SOP) as pretraining tasks [19].

Finetuning Data. We finetune and evaluate SqueezeBERT (and other baselines) on the General Language Understanding Evaluation (GLUE) set of tasks. This benchmark consists of a diverse set of 9 NLU tasks; thanks to the structure and breadth of these tasks (see supplementary material for detailed task-level information), GLUE has become the standard evaluation benchmark for NLP research. A model's performance across the GLUE tasks likely provides a good approximation of that model's generalizability (esp. to other text classification tasks).

4.2 Training Methodology

Many of the recent papers on efficient NLP networks report results on models trained with bells and whistles such as distillation, adversarial training, and/or transfer learning across GLUE tasks. However, there is no standardization of these training schemes across different papers, making it difficult to distinguish the contribution of the model from the contribution of the training scheme to the final accuracy number. Therefore, we first train SqueezeBERT using a simple training scheme (described in Section 4.2.1, with results reported in Section 5.1), and then we train SqueezeBERT with distillation and other techniques (described in Section 4.2.2, with results reported in Section 5.2).

4.2.1 Training without bells and whistles

We pretrain SqueezeBERT from scratch (without distillation) using the LAMB optimizer, and we employ the hyperparameters recommended by the LAMB authors: a global batch size of 8192, a learning rate of 2.5e-3, and a warmup proportion of 0.28 [38]. Following the LAMB paper's recommendations, we pretrain for 56k steps with a maximum sequence length of 128 and then for 6k steps with a maximum sequence length of 512.

For finetuning, we use the AdamW optimizer with a batch size of 16 without momentum or weight decay with $\beta_1 = 0.9$ and $\beta_2 = 0.999$ [39]. As is common in the literature, during finetuning for each task, we perform hyperparameter tuning on the learning rate and dropout rate. We present more details on this in the supplementary material. In the interest of a fair comparison, we also train BERT-base using the aforementioned pretraining and finetuning protocol.

4.2.2 Training with bells and whistles

We now review recent techniques for improving the training of NLP networks, and we describe the approaches that we will use for the training and evaluation of SqueezeBERT in Section 5.2.

Distillation approaches used in other efficient NLP networks. While the term "knowledge distillation" was coined by Hinton *et al.* to describe a specific method and equation [40], the term "distillation" is now used in reference to a diverse range of approaches where a "student" network is trained to replicate a "teacher" network. Some researchers distill only the final layer of the network [17], while others also distill the hidden layers [15, 18, 22]. When distilling the hidden layers, some apply layer-by-layer distillation warmup, where each module of the student network is distilled

Table 2: Comparison of neural networks on the **development** set of the GLUE benchmark. †denotes models trained by the authors of the present paper. Bells and whistles are: *A* = adversarial training; *D* = distillation of final layer; *E* = distillation of encoder layers; *S* = transfer learning across GLUE tasks (a.k.a. STILTs [41]); *W* = per-layer warmup. In GLUE accuracy, a dash means that accuracy for this task is not provided in the literature.

		GLUE accuracy										efficiency			
Model	Bells & Whistles	MNLI-m	MNLI-mm	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	Average	#MParams	GFLOPs	Latency (ms)	Speedup
		*Results **without** bells and whistles*													
BERT-base†	-	85.2	84.8	89.9	92.2	92.7	62.8	90.7	91.2	76.5	85.1	109	22.5	1690	1.0x
MobileBERT [15]	-	80.8	-	-	88.2	90.1	-	-	84.3	-	-	25.3	5.36	572	3.0x
ALBERT-base [19]	-	81.6	-	-	-	90.3	-	-	-	-	-	12.0	22.5	1690	1.0x
SqueezeBERT†	-	82.3	82.9	89.4	90.5	92.0	53.7	89.4	89.8	71.8	82.4	51.1	7.42	390	4.3x
		*Results **with** bells and whistles*													
DistilBERT 6/768 [17]	D	82.2	-	88.5	89.2	91.3	51.3	86.9	87.5	59.9	-	66	11.3	814	2.1x
Turc 6/768 [20]	D	82.5	83.4	89.6	89.4	91.1	-	-	87.2	66.7	-	67.5	11.3	814	2.1x
Theseus 6/768 [22]	DESW	82.3	-	89.6	89.5	91.5	51.1	88.7	89.0	68.2	-	66	11.3	814	2.1x
MobileBERT [15]	DEW	84.4	-	-	91.5	92.5	-	-	87.0	-	-	25.3	5.36	572	3.0x
SqueezeBERT†	DS	82.5	82.9	89.5	90.9	92.2	53.7	90.3	92.0	80.9	84.0	51.1	7.42	390	4.3x

independently while downstream modules are frozen [15]. Some distill during pretraining [15, 17], some distill during finetuning [22], and some do both [18, 21].

Bells and whistles used for training SqueezeBERT (for results in Section 5.2). Distillation is not a central focus of this paper, and there is a large design space of potential approaches to distillation, so we select a relatively simple form of distillation for use in SqueezeBERT training. We apply distillation only to the final layer, and only during finetuning. On the GLUE sentence classification tasks, we use soft cross entropy loss with respect to a weighted sum of the teacher's logits and a one-hot encoding of the ground-truth. Also note that GLUE has one regression task (STS-B text similarity), and for this task we replace the soft cross entropy loss with mean squared error. In addition to distillation, inspired by STILTS [41] and ELECTRA [12] we apply transfer learning from the MNLI GLUE task to other GLUE tasks as follows. The SqueezeBERT student model is pretrained using the approach described in Section 4.2.1, and then it is finetuned on the MNLI task. The weights from MNLI training are used as the initial student weights for other GLUE tasks except for CoLA.[6] Similarly, the teacher model is a BERT-base model that is pretrained using the ELECTRA method and then finetuned on MNLI. Then, the teacher model is finetuned independently on each GLUE task, and these task-specific teacher weights are used for distillation.

5 Results

We now turn our attention to comparing SqueezeBERT to other efficient neural networks.

5.1 Results *without* bells and whistles

In the upper portions of Tables 2 and 3, we compare our results to other efficient networks on the dev and test sets of the GLUE benchmark. Note that relatively few of the efficiency-optimized networks report results without bells and whistles, and most such results are reported on the development (not test) set of GLUE. Fortunately, the authors of MobileBERT – a network which we will find in the next section compares favorably to other efficient networks with bells and whistles enabled – do

[6] For CoLA, the student weights are pretrained (per Section 4.2.1) but not finetuned on MNLI prior to task-specific training.

Table 3: Comparison of neural networks on the **test** set of the GLUE benchmark. †denotes models trained by the authors of the present paper. Bells and whistles are: A = adversarial training; D = distillation of final layer; E = distillation of encoder layers; S = transfer learning across GLUE tasks (a.k.a. STILTs [41]); W = per-layer warmup.

		GLUE accuracy											efficiency			
Model	Bells & Whistles	MNLI-m	MNLI-mm	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	WNLI	GLUE score	#MParams	GFLOPs	Latency (ms)	Speedup
		*Results **without** bells and whistles*														
BERT-base†	-	84.4	84.2	80.5	91.4	92.8	51.3	86.9	87.9	70.7	65.1	79.0	109	22.5	1690	1.0x
BERT-base [9]	-	84.6	83.4	80.2	90.5	93.5	52.1	86.5	86.9	66.4	65.1	78.3	109	22.5	1690	1.0x
SqueezeBERT†	-	82.0	81.1	80.1	90.1	91.0	46.5	84.9	86.1	66.7	65.1	76.9	51.1	7.42	390	4.3x
		*Results **with** bells and whistles*														
TinyBERT 4/312 [21]	DE	82.5	81.8	-	87.7	92.6	43.3	79.9	-	62.9	65.1	-	14.5	1.2	118	14x
ELECTRA-Small++ [12]	AS	81.6	-	-	88.3	91.1	55.6	84.6	84.9	63.6	65.1	-	14.0	2.62	248	6.8x
PKD 6/768 [18]	DE	81.5	81.0	79.8	89.0	92.0	-	-	82.5	-	65.1	-	67.0	11.3	814	2.1x
Turc 6/768 [20]	D	82.8	82.2	79.7	89.4	91.8	-	-	84.3	65.3	65.1	-	67.5	11.3	814	2.1x
Theseus 6/768 [22]	DESW	82.4	82.1	80.5	89.6	92.2	47.8	84.9	85.4	66.2	65.1	77.1	66	11.3	814	2.1x
MobileBERT [15]	DEW	84.3	83.4	79.4	91.6	92.6	51.1	85.5	86.7	70.4	65.1	78.5	25.3	5.36	572	3.0x
SqueezeBERT†	DS	82.0	81.1	80.3	90.1	91.4	46.5	86.7	87.8	73.2	65.1	78.1	51.1	7.42	390	4.3x

provide development-set results without distillation on 4 of the GLUE tasks.[7] We observe in the upper portion of Table 2 that, when both networks are trained without distillation, SqueezeBERT achieves higher accuracy than MobileBERT on all of these tasks. This provides initial evidence that the techniques from computer vision that we have adopted can be applied to NLP, and reasonable accuracy can be obtained. Further, we observe that SqueezeBERT is 4.3x faster than BERT-base, while MobileBERT is 3.0x faster than BERT-base.[8]

Due to the dearth of efficient neural network results on GLUE without bells and whistles, we also provide a comparison in Table 2 with the ALBERT-base network. ALBERT-base is a version of BERT-base that uses the same weights across multiple attention layers, and it has a smaller encoder than BERT. Due to these design choices, ALBERT-base has 9x fewer parameters than BERT-base. However, ALBERT-base and BERT-base have the same number of FLOPs, and we observe in our measurements in Table 2 that ALBERT-base does not offer a speedup over BERT-base on a smartphone.[9] Further, on the two GLUE tasks where the ALBERT authors reported the accuracy of ALBERT-base, MobileBERT and SqueezeBERT both outperform the accuracy of ALBERT-base.

5.2 Results *with* bells and whistles

Now, we turn our attention to comparing SqueezeBERT to other models, all trained with bells-and-whistles. In the lower portion of Table 3, we first observe that when trained with bells-and-whistles MobileBERT matches or outperforms the accuracy of the other efficient models (except SqueezeBERT) on 8 of the 9 GLUE tasks. Further, on 4 of the 9 tasks SqueezeBERT outperforms the accuracy of MobileBERT; on 4 of 9 tasks MobileBERT outperforms SqueezeBERT; and on 1 task (WNLI) all models predict the most frequently occurring category.[10] Also, SqueezeBERT achieves

[7] Note that some papers report results on only the development set or the test set, and some papers only report results on a subset of GLUE tasks. Our aim with this evaluation is to be as inclusive as possible, so we include papers with incomplete GLUE results in our results tables.

[8] In our measurements, we find MobileBERT takes 572ms to classify one length-128 sequence on a Pixel 3 phone. This is slightly faster than the 620ms reported by the MobileBERT authors in the same setting [42]. We use the faster number in our comparisons. Further, all latencies in our results tables were benchmarked by us.

[9] However, reducing the number of parameters while retaining a high number of FLOPs can present other advantages, such as faster distributed training [19, 43] and superior energy-efficiency [44].

[10] Note that data augmentation approaches have been proposed to improve accuracy on WNLI; see [45]. For fairness in comparing against our baselines, we choose not to use data augmentation to improve WNLI results.

an average score across all GLUE tasks that is within 0.4 percentage-points of MobileBERT. Given the speedup of SqueezeBERT over MobileBERT, we think it is reasonable to say that SqueezeBERT and MobileBERT each offer a compelling speed-accuracy tradeoff for NLP inference on mobile devices.

6 Related Work

Quantization and Pruning. Quantization is a family of techniques which aims to reduce the number of bits required to store each parameter and/or activation in a neural network, while at the same time maintaining the accuracy of that network. This has been successfully applied to NLP in such works as [46, 47]. Pruning aims to directly eliminate certain parameters from the network while also maintaining accuracy, thereby reducing the storage and potentially computational cost of that network; for an application of this to NLP, please see [48]. These methods could be applied to SqueezeBERT to yield further efficiency improvements, but quantization and pruning are not a focus of this paper.

Convolutions in self-attention networks for language-generation tasks. In this paper, our experiments focus on natural language understanding (NLU) tasks such as sentence classification. However, another widely-studied area is natural language generation (NLG), which includes the tasks of machine-translation (e.g., English-to-German) and language modeling (e.g., automated sentence-completion). While we are not aware of work that adopts convolutions in self-attention networks for NLU, we *are* aware of such work in NLG. For instance, the Evolved Transformer and Lite Transformer architectures contain self-attention modules and convolutions in separate portions of the network [49, 50]. Additionally, LightConv shows that well-designed convolutional networks without self-attention produce comparable results to self-attention networks on certain NLG tasks [51]. Also, Wang *et al.* sparsify the self-attention matrix multiplication using a pattern of nonzeros that is inspired by dilated convolutions [52]. Finally, while not an attention network, Kim applied convolutional networks to NLU several years before the development of multi-head self-attention [53].

7 Conclusions & Future Work

In this paper, we have studied how grouped convolutions, a popular technique in the design of efficient CV neural networks, can be applied to NLP. First, we showed that the position-wise fully-connected layers of self-attention networks can be implemented with mathematically-equivalent 1D convolutions. Further, we proposed SqueezeBERT, an efficient NLP model which implements most of the layers of its self-attention encoder with 1D grouped convolutions. This model yields an appreciable >4x latency decrease over BERT-base when benchmarked on a Pixel 3 phone. We also successfully applied distillation to improve our approach's accuracy to a level that is competitive with a distillation-trained MobileBERT and with the original version of BERT-base.

We now discuss some possibilities for future work in the directions outlined in this paper. There are several techniques in use in CV that could be applied to NLP which we have not covered in this paper. One very promising direction is downsampling strategies which decrease the sequence length of the activations in the self-attention network as the layers progress. Extensions of this idea, such as U-Nets [54], as well as modifying channel sizes (hidden size) instead of and in addition to sequence length would also be promising directions. On this path of techniques, applying ideas such as BiFPNs [55], striding, and dilation [56] may also yield interesting results.

The addition of all of these potential techniques opens up a significantly broader search-space of neural architecture designs for NLP. This motivates the application of automated neural architecture search (NAS) approaches such as those described in [57, 58].

Broader Impact

Potential benefits of this work

We hope the techniques used in this paper will allow more efficient and practical deployment of self-attention based networks, particularly allowing more widespread use of these networks on mobile devices. Possible use cases include email sorting, chat analysis, spam detection, or hate-speech filtering. Facebook has begun using a self-attention based network to automatically detect and remove hate speech on their platform [59, 60]. Currently, companies usually run these models on the server-side, but some may be reticent to deploy them on a mass scale due to computation costs. Mobile inference wouldn't require expensive server infrastructure and allows use in cases where privacy and security may be a concern. Running on the edge devices may allow more real-time or offline use cases such as grammar checking, or sentence completion.

Potential malicious uses of this work

Given that we are releasing our model and training and inference code as free software, anyone can train our model on any dataset that they like. While we hope most practitioners will apply our work for altruistic or at least well-intentioned purposes, some may apply our work for ethically questionable or purely self-serving applications. For instance, low-cost mobile inference could allow "smart" key-loggers and eavesdropping viruses to be deployed, and for these devices to better avoid detection by computing locally and only uploading important information. And, while we hope that social networks and other text-centric portals will draw on our work to embed fair and just models into their mobile applications for content moderation, these companies could just as easily draw on our work to train and deploy models that censor content of political enemies or amplify only certain types of messages and voices.

Potential effects of unintended bias in the neural network and its training data

Models for NLP commonly suffer from biases regarding race and gender [61]. In our work, we have used standard datasets, and we are not aware of any experimental factors that would increase or decrease SqueezeBERT's propensity for bias, as compared to the approaches used in similar self-attention research such as BERT and ELECTRA. The gender-related biases of our pretraining corpora (Wikipedia and BooksCorpus) are investigated by Tan *et al.* [61], and the gender biases of BERT and GPT models finetuned on the GLUE tasks are investigated by Babaeianjelodar *et al.* [62].

Aside from gender bias, we are not aware of studies that investigate other patterns of bias in the datasets that we used in our work. However, Sheng *et al.* investigate biases regarding race, gender, and sexual orientation in a GPT-2 self-attention model trained on a language-modeling (text-generation) task [63]. Further, in the paper introducing the GPT-3 model, a study similar to that of Sheng *et al.* is performed [64].

According to the taxonomy described in [65], the *harms* caused by biased NLP technology include:

- Allocational harms: These arise when an "automated system allocates resources (e.g., credit) or opportunities (e.g., jobs) unfairly to different social group."
- Representational harms: These arise when "a system (e.g., a search engine) represents some social groups in a less favorable light than others, demeans them, or fails to recognize their existence altogether."

We direct the interested reader to work by Blodgett *et al.* [65] and Sun *et al.* [66] for more details and suggestions on how to minimize the bias in datasets and the neural networks that learn from them.

Acknowledgments and Disclosure of Funding

K. Keutzer's research is supported by Alibaba, Amazon, Google, Facebook, Intel, and Samsung.

References

[1] D. Sayce, "The number of tweets per day in 2019," https://www.dsayce.com/social-media/tweets-day/, 2019.

[2] J. Schultz, "How much data is created on the internet each day?" https://blog.microfocus.com/how-much-data-is-created-on-the-internet-each-day/, 2019.

[3] A. Al-Heeti, "WhatsApp: 65B messages sent each day, and more than 2B minutes of calls," *CNET*, 2018, https://www.cnet.com/news/whatsapp-65-billion-messages-sent-each-day-and-more-than-2-billion-minutes-of-calls/.

[4] Templatify, "How many emails are sent every day? top email statistics for business," https://info.templafy.com/blog/how-many-emails-are-sent-every-day-top-email-statistics-your-business-needs-to-know, 2017.

[5] Lovely Mobile News, "Mobile has largely displaced other channels for email," https://lovelymobile.news/mobile-has-largely-displaced-other-channels-for-email, 2017.

[6] G. Donnelly, "75 super-useful facebook statistics for 2018," https://www.wordstream.com/blog/ws/2017/11/07/facebook-statistics, 2018.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Conference on Neural Information Processing Systems (NeurIPS)*, 2017.

[8] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_pap 2018.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.

[10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," https://d4mucfpksywv.cloudfront.net/better-language-models/language_models_are_unsupervised_multitask_learners.pd 2019.

[11] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," 2019.

[12] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training text encoders as discriminators rather than generators," in *International Conference on Learning Representations (ICLR)*, 2020.

[13] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized bert pretraining approach," *arXiv:1907.11692*, 2019.

[14] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: Alexnet-level accuracy with 50x fewer parameters and <0.5mb model size," *arXiv:1602.07360*, 2016.

[15] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: a compact task-agnostic BERT for resource-limited devices," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, arXiv:2004.02984. [Online]. Available: https://arxiv.org/abs/2004.02984

[16] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," *arXiv:1804.07461*, 2018.

[17] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.

[18] S. Sun, Y. Cheng, Z. Gan, and J. Liu, "Patient knowledge distillation for BERT model compression," in *Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[19] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite bert for self-supervised learning of language representations," in *ICLR*, 2019. 21

[20] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv:1908.08962*, 2019. 15

[21] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "TinyBERT: Distilling bert for natural language understanding," *arXiv:1909.10351*, 2019. 13

[22] C. Xu, W. Zhou, T. Ge, F. Wei, and M. Zhou, "Bert-of-theseus: Compressing bert by progressive module replacing," *arXiv:2002.02925*, 2020. 22

1

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 18

[24] K. Fukushima, "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position," *Biological Cybernetics*, 1980. 14

[25] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, 1989. 16

[26] A. Krizhevsky *et al.*, "cuda-convnet," 2011, https://code.google.com/archive/p/cuda-convnet. 11

[27] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *NeurIPS*, 2012. 9

[28] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, 2015. 2

[29] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, https://arxiv.org/abs/1610.02357. 7

[30] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 8

[31] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017. 10

[32] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *CVPR*, 2018. 6

[33] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*, 2019. [Online]. Available: http://proceedings.mlr.press/v97/tan19a.html 3

[34] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *NeurIPS*, 2019. 17

[35] M. Schuster and K. Nakajima, "Japanese and korean voice search," in *International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2012. 12

[36] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, Łukasz Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. Corrado, M. Hughes, and J. Dean, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv:1609.08144*, 2016. 20 19

[37] Y. Zhu, R. Kiros, R. Zemel, R. Salakhutdinov, R. Urtasun, A. Torralba, and S. Fidler, "Aligning books and movies: Towards story-like visual explanations by watching movies and reading books," in *IEEE International Conference on Computer Vision (ICCV)*, 2015. 4

[38] Y. You, J. Li, S. Reddi, J. Hseu, S. Kumar, S. Bhojanapalli, X. Song, J. Demmel, K. Keutzer, and C.-J. Hsieh, "Large batch optimization for deep learning: Training bert in 76 minutes," in *ICLR*, 2020. 5

[39] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *ICLR*, 2019.

[40] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv:1503.02531*, 2015.

[41] J. Phang, T. Févry, and S. R. Bowman, "Sentence encoders on stilts: Supplementary training on intermediate labeled-data tasks," *arXiv:1811.01088*, 2018.

[42] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: Task-agnostic compression of BERT by progressive knowledge transfer," *OpenReview submission*, 2019. [Online]. Available: https://openreview.net/forum?id=SJxjVaNKwB

[43] F. N. Iandola, K. Ashraf, M. W. Moskewicz, and K. Keutzer, "FireCaffe: near-linear acceleration of deep neural network training on compute clusters," in *CVPR*, 2016.

[44] F. Iandola and K. Keutzer, "Small neural nets are beautiful: Enabling embedded systems with small deep-neural-network architectures," in *ESWEEK Keynote*, 2017.

[45] V. Kocijan, A.-M. Cretu, O.-M. Camburu, Y. Yordanov, and T. Lukasiewicz, "A surprisingly robust trick for winograd schema challenge," in *ACL*, 2019.

[46] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Q-BERT: Hessian based ultra low precision quantization of bert," in *AAAI*, 2020.

[47] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, "Q8BERT: Quantized 8bit bert," *arXiv:1910.06188*, 2019.

[48] V. Sanh, T. Wolf, and A. M. Rush, "Movement pruning: Adaptive sparsity by fine-tuning," *arXiv:2005.07683*, 2020.

[49] D. R. So, C. Liang, and Q. V. Le, "The evolved transformer," in *ICLR*, 2019

[50] Z. Wu, Z. Liu, J. Lin, Y. Lin, and S. Han, "Lite transformer with long short term attention," in *ICLR*, 2020.

[51] F. Wu, A. Fan, A. Baevski, Y. N. Dauphin, and M. Auli, "Pay less attention with lightweight and dynamic convolutions," in *ICLR*, 2019.

[52] C. Wang, Z. Ye, A. Zhang, Z. Zhang, and A. J. Smola, "Transformer on a diet," *arXiv:2002.06170*, 2020.

[53] Y. Kim, "Convolutional neural networks for sentence classification," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014. [Online]. Available: https://www.aclweb.org/anthology/D14-1181

[54] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*, 2015.

[55] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *CVPR*, 2020. [Online]. Available: https://arxiv.org/abs/1911.09070

[56] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *ICLR*, 2016.

[57] A. Shaw, D. Hunter, F. Iandola, and S. Sidhu, "SqueezeNAS: Fast neural architecture search for faster semantic segmentation," in *ICCV Neural Architects Workshop*, 2019.

[58] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "FBNet: Hardware-aware efficient convnet design via differentiable neural architecture search," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[59] R. Dansby, H. Ma, U. Ozertem, V. Stoyanov, F. Yang, H. Fang, C. Moghbel, X. Peng, S. Wang, K. Zhang *et al.*, "AI advances to better detect hate speech," *Facebook AI Blog*, 2020. [Online]. Available: https://ai.facebook.com/blog/ai-advances-to-better-detect-hate-speech

[60] K. Johnson, "Facebook is using more AI to detect hate speech," *VentureBeat*, 2020. [Online]. Available: https://venturebeat.com/2020/05/12/facebook-is-using-more-ai-to-detect-hate-speech

[61] Y. C. Tan and L. E. Celis, "Assessing social and intersectional biases in contextualized word representations," in *NeurIPS*, 2019.

[62] M. Babaeianjelodar, S. Lorenz, J. Gordon, J. Matthews, and E. Freitag, "Quantifying gender bias in different corpora," in *WWW Companion*, 2020.

[63] E. Sheng, K.-W. Chang, P. Natarajan, and N. Peng, "The woman worked as a babysitter: On biases in language generation," in *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019. [Online]. Available: https://www.aclweb.org/anthology/D19-1339 12

[64] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *arXiv:2005.14165*, 2020. 11 1

[65] S. L. Blodgett, S. Barocas, H. Daume III, and H. Wallach, "Language (technology) is power: A critical survey of "bias" in nlp," *arXiv:2005.14050*, 2020. 16

[66] T. Sun, A. Gaut, S. Tang, Y. Huang, M. ElSherief, J. Zhao, D. Mirza, E. Belding, K.-W. Chang, and W. Y. Wang, "Mitigating gender bias in natural language processing: Literature review," in *Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2019. [Online]. Available: https://www.aclweb.org/anthology/P19-1159 7

[67] C. J. Van Rijsbergen, *Information Retrieval (2nd ed.)*. Butterworth-Heinemann, 1979. 13

[68] B. W. Matthews, "Comparison of the predicted and observed secondary structure of t4 phage lysozyme," *Biochimica et Biophysica Acta (BBA)-Protein Structure*, 1975. 9

[69] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018. 3

[70] Quora, "Quora question pairs," 2017. [Online]. Available: https://www.kaggle.com/c/quora-question-pairs/overview 5

[71] Z. Chen, H. Zhang, X. Zhang, and L. Zhao, "Quora question pairs," 2018. [Online]. Available: http://static.hongbozhang.me/doc/STAT_441_Report.pdf 19

[72] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *EMNLP*, 2016. 6

[73] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *EMNLP*, 2013. 4

[74] A. Warstadt, A. Singh, and S. R. Bowman, "Neural network acceptability judgments," in *Transactions of the Association for Computational Linguistics*, 2019. 14

[75] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation," in *Eleventh International Workshop on Semantic Evaluations*, 2017. 2

[76] W. B. Dolan and C. Brockett, "Automatically constructing a corpus of sentential paraphrases," in *Proceedings of the International Workshop on Paraphrasing*, 2005. 21

[77] I. Dagan, O. Glickman, and B. Magnini, "The pascal recognising textual entailment challenge," in *Machine Learning Challenges Workshop*. Springer, 2005, pp. 177–190. 15

[78] R. B. Haim, I. Dagan, B. Dolan, L. Ferro, D. Giampiccolo, B. Magnini, and I. Szpektor, "The second pascal recognising textual entailment challenge," in *Proceedings of the Second PASCAL Challenges Workshop on Recognising Textual Entailment*, 2006. 18

[79] D. Giampiccolo, B. Magnini, I. Dagan, and B. Dolan, "The third pascal recognizing textual entailment challenge," in *Proceedings of the ACL-PASCAL workshop on textual entailment and paraphrasing*. Association for Computational Linguistics, 2007, pp. 1–9. 8

[80] L. Bentivogli, I. Dagan, H. T. Dang, D. Giampiccolo, and B. Magnini, "The fifth pascal recognizing textual entailment challenge," 2009. 20

[81] H. J. Levesque, E. Davis, and L. Morgenstern, "The winograd schema challenge," in *Proceedings of the Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning*, 2012. 17

[82] A. W. Services, "Amazon s3 pricing," 2020. [Online]. Available: https://aws.amazon.com/s3/pricing 10

[83] ——, "Amazon ec2 p3 instance product details," 2020. [Online]. Available: https://aws.amazon.com/ec2/instance-types/p3 1 9

[84] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, and J. Brew, "Huggingface's transformers: State-of-the-art natural language processing," *arXiv:1910.03771*, 2019. 12

[85] NVIDIA, "Deep learning examples for tensor cores," 2020, https://github.com/NVIDIA/DeepLearningExamples. 16

[86] ——, "APEX - A PyTorch Extension: Tools for easy mixed precision and distributed training in pytorch," 2020, https://github.com/NVIDIA/apex. 13

[87] T. Do, "List of smartphones powered by qualcomm snapdragon 845 processor," *Techwalls*, 2019, https://www.techwalls.com/qualcomm-snapdragon-845-smartphones. 15

[88] H. Touvron, A. Vedaldi, M. Douze, and H. Jégou, "Fixing the train-test resolution discrepancy: FixEfficientNet," 2020. 11

[89] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014. 3

[90] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 2

[91] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 4

[92] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015. 8

[93] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *European Conference on Computer Vision (ECCV)*, 2018. 5

[94] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 14

[95] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv:1409.1556*, 2014. 6

[96] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," *arXiv:1409.4842*, 2014. 7

[97] X. Liu, P. He, W. Chen, and J. Gao, "Multi-task deep neural networks for natural language understanding," in *ACL*, 2019. 10

Supplementary Material for SqueezeBERT

A Overview of GLUE tasks

In this section we provide an overview of each of the tasks within the GLUE benchmark, their evaluation metrics, and their potential applications outside of the benchmark. For further information on the benchmark, please see [16], where it was originally proposed.

Note that some tasks have one evaluation metric, and some have two. The possible evaluation metrics are Accuracy (abbreviated below as `acc`), F1 [67], Matthews Correlation Coefficient (`MCC`) [68], Pearson Correlation (`pearson`), and Spearman Correlation (`spearman`).

MNLI: Multi-genre Natural Language Inference [69]. Given a pair of sentences (sentence 1 and sentence 2), the task is to predict whether sentence 2 entails sentence 1. Note that there are 2 test sets for this task: MNLI-matched (MNLI-m), and MNLI-mismatched (MNLI-mm). When computing an average score overall tasks, we follow the convention employed by the GLUE leaderboard, which is `(acc(MNLI-m) + acc(MNLI-mm))/2`. Therefore, while MNLI-m and MNLI-mm are two columns in our results table, they only count as one task in the average GLUE score. Potential applications include helping users improve their writing by checking for repetitive sections based on whether certain sentences are entailed by other previous ones. This task contains 392,703 training examples.

QQP: Quora Question Pairs [70, 71]. The goal of this task is to determine whether a pair of questions have the same meaning. In practical applications, such as organizing emails or organizing helpdesk tickets, this approach can be used to group similar questions together and only answer them once. There are 2 metrics for this task: `acc` and `F1`. In our results tables, we follow the approach of the GLUE leaderboard and compute QQP results as the average of these two metrics. We observe that, for most models, on the test set the F1 score is significantly lower than the accuracy score. For instance, previously reported BERT-base results are F1=71.2 and accuracy=89.2. The TinyBERT paper [21] reports a TinyBERT QQP score of 71.3, relative to a BERT-base baseline of 71.2, so it appears that the TinyBERT paper only reports the QQP F1 (but not accuracy) score. Conversely, the ELECTRA [12] paper reports an ELECTRA-small QQP score of 88.0, relative to a BERT-base QQP score of 89.2, so it appears that the ELECTRA paper only reports the QQP accuracy (but not F1) score. Out of fairness, we have omitted the TinyBERT and ELECTRA-small QQP test-set results from our results table, as it appears that they are each using a metric different from our metric of `((acc(QQP) + F1(QQP))/2)`. This task contains 363,871 training examples.

QNLI: Question-answering Natural Language Inference. This task is derived from the dataset published in [72]. Specifically, it is reformulated as a two-class classification task, wherein the goal is to decide whether or not the given sentence does, or does not, contain the answer to a given question. In this way, QNLI bears some similarity to other entailment classification tasks such as MNLI. The evaluation metric used for QNLI is just `acc(QNLI)`. In terms of applications, QNLI is useful for many of the same situations in which a standard question-answering dataset would be, such as in designing intelligent voice assistants which can respond to user questions. For instance, if an extractive or generative model is used to generate an answer to a question, a QNLI-trained model could be used as a sanity-check on whether the machine-generated response actually answers the question. Also, a QNLI-trained model could be used to check if an existing answer already answers a new question on a form or in a helpdesk database, thereby not requiring a new answer to be generated by a human. This task contains 104,744 training examples.

SST-2: Stanford Sentiment Treebank [73]. This is a sentiment classification dataset, with the goal being to predict whether a sentence has a positive or negative sentiment. The source data for SST-2 is hand-annotated movie review data. The metric used for this task is `acc(SST-2)`. SST-2 applications include improving chatbot based systems which require responses that properly match the conversation context to be generated, and potentially triaging emails at helpdesks so that, for example, the ones with strong negative sentiments are prioritized. This task contains 67,350 training examples.

CoLA: Corpus of Linguistic Acceptability [74]. This is a binary classification task, where the objective is to determine whether a given sentence meets the criteria for being a properly written English sentence. It is worth noting that CoLA is a binary classification task, and 69% of the data samples in the validation set are positive examples. The metric used for this task is different from the metrics used in other GLUE tasks: `MCC(CoLA)`. A distinguishing feature of Matthews Correlation Coefficient (`MCC`) is that it effectively adjusts for class imbalance in the dataset. On the

CoLA validation set, where 69% of the samples are positive, predicting the majority class would produce an accuracy of 69% but an `MCC` of 0.0. Also note that `MCC` can be anywhere between and including -1.0 and 1.0, as opposed to 0.0 (0%) and 1.0 (100%) for `acc`. Models trained on CoLA could be used to provide feedback on the grammatical correctness of user-generated text, analogous to a spell-checker, but potentially with more nuanced understanding of grammar than a rule-based grammar checker. CoLA-trained models could also be used to verify the grammatical correctness of text generated by other neural networks. This task contains 8,551 training examples.

STS-B: Semantic Textual Similarity Benchmark [75]. This dataset is built from several sources including news headlines data as well as captions data. The goal of this task is to predict the level of semantic similarity between two input sentences, on a scale of 1 (minimum similarity) to 5 (maximum similarity). Notably, this is a regression task. The overall evaluation metric used on the GLUE leaderboard, which we report in our results tables, is `(spearman(STS-B) + pearson(STS-B))/2`. STS-B could have applications in several areas, such as plagiarism detection, as well as potentially other applications which are mentioned above in the description of QQP. This task contains 5,750 training examples.

MRPC: Microsoft Research Paraphrase Corpus [76]. This is classification task where the goal is to predict whether or not the two input sentences have the same semantic meaning. The data itself comes from online news articles and is hand-annotated. The metric used for this task is `((acc(QQP) + F1(QQP))/2)`, consistent with the GLUE test set leaderboard. MRPC would have applications similar to STS-B, such as plagiarism detection and others. This task contains 3,669 training examples.

RTE: Recognizing Textual Entailment is a combination of several datasets from yearly challenges on entailment [77, 78, 79, 80]. RTE is a two-way classification task where the labels are *entailment* and *not_entailment*. The metric used for this task is `acc(RTE)`. Potential applications of this dataset are similar to those of MNLI. This task contains 2,491 training examples.

WNLI: Winograd Schema Challenge, reformulated as Winograd Natural Language Inference [81]. This goal of this task is for the model to predict the antecedent of a pronoun (out of a set of choices) given the input sentence containing that pronoun. The metric used for this task is `acc(WNLI)`. As with other work in this area [9], we simply predict the majority class on this dataset, resulting in a test set accuracy of 65.1%. This task contains 636 training examples.

In Listings 1 and 2, we show how we calculate overall scores for the GLUE development set and test set. Other than omitting WNLI from the development-set score (following [9]), these calculations adhere to the methodology used by the GLUE leaderboard.

Listing 1: Calculating average score on the GLUE **development** set.

```
dev_score = MEAN(
                 (acc(MNLI-m) + acc(MNLI-mm))/2,
                 (F1(QQP) + acc(QQP))/2,
                 acc(QNLI),
                 acc(SST-2),
                 MCC(CoLA),
                 (pearson(STS-B) + spearman(STS-B))/2,
                 (F1(MRPC) + acc(MRPC))/2,
                 acc(RTE)
                )
```

Listing 2: Calculating average score on the GLUE **test** set.

```
test_score = MEAN(
                  (acc(MNLI-m) + acc(MNLI-mm))/2,
                  (F1(QQP) + acc(QQP))/2,
                  acc(QNLI),
                  acc(SST-2),
                  MCC(CoLA),
                  (pearson(STS-B) + spearman(STS-B))/2,
                  (F1(MRPC) + acc(MRPC))/2,
                  acc(RTE),
                  acc(WNLI)
                 )
```

B More training details and results

B.1 Training Hardware

We do all pretraining and finetuning on an 8-GPU server without multi-server distributed training. Our server has 8 NVIDIA Titan RTX GPUs. The server also has an Intel Xeon Gold 6130 64-core CPU, and it has 256GB of RAM. Further, the data loading requirements for our NLP application are so small that we were able to use low-cost, low-bandwidth spinning-media (not flash) disks for storing the training data. We employed two of these servers for approximately 6 months to do various experiments, beginning with reproducing BERT-base, and then experimenting with various model architectures. No cloud computing resources or corporate computing resources were used for this research.

While we used our own computing resources for this work, we now consider what it *would have cost* to rent servers similar to ours from a cloud provider for the duration of our project. We can break this down into two types of costs, *storage* and *computation*:

- **Storage.** The datasets are small (under 100GB total), and we typically retain a few hundred gigabytes of model parameters from various training checkpoints, so we imagine the cloud storage costs would be negligible: As of this writing, Amazon Web Services charges $0.023/GB for general-purpose storage [82], so storing a terabyte of data and model checkpoints would cost just $23 per month, or $138 for 6 months. There may be additional fees for data transfers within the AWS ecosystem, but we have studied these costs in detail.
- **Computation.** As we mentioned earlier, to do the work in this project, including the initial work to reproduce baselines and the work to experiment with several potential neural network designs, we used 2 8-GPU servers for about 6 months. We estimate that renting this amount of computation from a cloud provider would cost around $100,000, calculated as (6 months) * (50% utilization) * (2 servers) * (30 days per month) * (24 hrs/day) * ($24/hr for an 8-GPU V100 machine from Amazon Web Services [83]).

So, while we used our computing resources for this work, doing this entire research project from beginning to end in the cloud would have cost around $100,000.

Now, we consider what it would cost to reproduce the SqueezeBERT model from scratch using cloud hardware. The SqueezeBERT model can be reproduced in approximately 5 days: 4 days for pretraining, and then under one day for finetuning on all GLUE tasks with the optimal hyperparameters discovered by our hyperparameter search (see Section B.4). In total, this would cost approximately $2880, calculated as (5 days) * (24 hrs/day) * ($24/hr for one 8-V100 AWS machine).

B.2 Training Software

Our PyTorch-based training and inference code draws heavily on the HuggingFace Transformers [84] and NVIDIA Deep Learning Examples [85] repositories. We perform pretraining using 8 GPUs with 16-bit floating-point math, and we use the O2 optimization level in the NVIDIA Apex mixed-precision training primitives [86]. We perform finetuning on a single GPU with 32-bit floating-point math, and we concurrently run multiple finetuning tasks across the 8-GPU machine.

B.3 Further details: distillation

In the distillation approach that we described in Section 4.2.2, we mentioned that we use teacher logits and one-hot ground-truth as the target output for our soft cross entropy loss. The weighting between the teacher logits and the ground-truth is controlled by a hyperparameter α. Let Ψ_t represent the teacher logits and let Ψ_g represent the one-hot encoding of the ground-truth. Formally, we write this weighted sum as:

$$\Psi = (1 - \alpha)\Psi_t + \alpha\Psi_g \tag{4}$$

In the next section, we will tune α as part of our hyperparameter tuning scheme.

B.4 Details of hyperparameter search during finetuning on GLUE tasks

We now present more details on the hyperparameter search approach that we used for training SqueezeBERT with bells and whistles. In Table 4, we present the space of possible hyperparameters over which we performed a grid-search. Note that the time to finetune the model using one

Table 4: Our hyperparameter search space.

Hyperparameter	MNLI, QQP, QNLI, STS-2	STS-B, MRPC, RTE	CoLA
α	[0.8, 0.9, 1.0]	[0.8, 0.9, 1.0]	[0.8, 0.9, 1.0]
Learning Rate	[1e-05, 2e-05, 3e-05, 4e-05]	[1e-05, 2e-05, 3e-05, 4e-05, 5e-05]	[1e-05, 2e-05, 3e-05, 4e-05, 5e-05]
Encoder Dropout	[0.0, 0.1]	[0.0, 0.1]	[0.0, 0.1]
Final Dropout	[0.1, 0.2]	[0.1, 0.2]	[0.0, 0.1]
Epochs	5	10	20
Batch Size	16	16	[16, 32, 48]

Table 5: Hyperparameters selected by our hyperparameter search when training SqueezeBERT with bells-and-whistles.

Hyperparameter	MNLI-m	MNLI-mm	QQP	QNLI	STS-2	CoLA	STS-B	MRPC	RTE
α	1.0	1.0	0.8	1.0	0.8	1.0	0.8	0.9	0.8
Learning Rate	3e-05	3e-05	4e-05	3e-05	3e-05	2e-05	4e-05	3e-05	3e-05
Encoder Dropout	0.0	0.0	0.1	0.0	0.1	0.0	0.1	0.1	0.1
Final Dropout	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
Epochs	4	4	5	2	5	5	10	9	3
Batch Size	16	16	16	16	16	16	16	16	16

set of hyperparameters varies significantly depending on the GLUE task, from 15 minutes for small datasets like RTE, to 14 hours for MNLI. For smaller datasets that require less training (e.g. RTE and MRPC), we use a broader search space and more epochs. And, for larger datasets (e.g. MNLI and QQP), we use a more narrow search space with fewer epochs.

Now, in Table 5, we present the best hyperparameters found in our search. We observe two interesting phenomena on this table. The first is regarding the use of distillation. Recall from Equation 4 that α is the hyperparameter that sets the weighting between the teacher logits and the ground-truth for distillation. When $\alpha = 1.0$, the teacher logits are ignored, and thus distillation is disabled. So, it is interesting to note that on three of the eight GLUE tasks in Table 5 (MNLI, QNLI, and CoLA), distillation did not produce superior results over non-distillation finetuning. The second interesting phenomenon in this table is that maximum accuracy was not necessarily achieved on final epoch. For example, on QNLI, SqueezeBERT converged to its maximum development-set accuracy after just two epochs.

C Further details: Inference on a smartphone

To evaluate inference speed, we run the neural networks on a Google Pixel 3 smartphone. This phone contains the popular Qualcomm Snapdragon 845 processing chip, which is also used in popular smartphones from Samsung, Xiaomi, and Sony [87]. The phone also has 4GB of LPDDR4x memory. To run a neural network on the phone, we do the following. First, we export the network to TorchScript using the `torch.jit.trace()` command, which yields a standalone neural net that does not require Python to run. Then, we copy the network to the phone, and we run it using a modified version of the `speed_benchmark_torch.cc` file that is built into PyTorch. We report the average latency of 40 runs, using the CPU cores of the phone. Following the protocol of MobileBERT, each run operates on a length-128 input sequence and a batch size of 1.

D More related work

Efficient convolutional networks for CV. Convolutional neural networks lead the state-of-the-art on computer vision (CV) tasks such as image classification, object detection, and semantic segmentation. In the last five years, the community has developed efficient convolutional neural networks for CV that run efficiently and in real-time on mobile devices. On the ImageNet [28] image classification task, from the year 2016 (ResNet-101 [23]) to 2020 (FixEfficientNet-D0 [88]), there is a 20x reduction in the number of floating-point operations (FLOPs) required, while accuracy has actually improved.[11][12] Further, on the COCO [89] object detection task, from 2016 (FPN [90]) to

[11] In our terminology throughout the paper, a multiply-add operation is two FLOPs.

[12] These results hold without expanding the training data beyond the ImageNet training set.

2020 (EfficientDet-D1 [55]), there is a 38x reduction in the number of FLOPs and an improvement in accuracy.[13] Finally, on the Cityscapes [91] semantic segmentation task, from 2016 (FCN-8s [92]) to 2020 (SqueezeNAS-MAC-Small [57]), there is a 334x reduction in the number of FLOPs, and an improvement in accuracy.[14] Given that the datasets were, for the most part, held constant, to what do we owe these improvements? One factor that has helped is the design of new neural network architectures with superior FLOP-accuracy tradeoffs. There have been several innovations to neural architecture for CV, including dilated convolutions [56], creative approaches for multiscale recognition [90, 55], and long-range aggregation of skip-connections [94]. However, above all else, there is one influential design element that has been adopted by nearly all efficient convolutional neural network designs developed in the last three years: grouped convolutions. Grouped convolutions, which are form of structured sparsity in the channel dimension, have a hyperparameter G, which reduces the number of parameters and the number of computations by $\frac{1}{G}$ Grouped convolutions were proposed by Krizhevsky *et al.* in 2012 [26, 27], but they were largely forgotten, and they were not used by subsequent state-of-the-art networks such as VGG [95], Inception [96], and ResNet [23]. Grouped convolutions began to reemerge in the literature in 2016 with Xception [29] and ResNext [30]. For ImageNet image classification, MobileNet showed in 2017 that in certain cases grouped convolutions can preserve most of the accuracy (compared to a non-grouped baseline) while producing extreme reductions in FLOPs, particularly when setting $G = C$ (where C is the number of channels) in certain layers [31].[15] The efficient FixEfficientNet, EfficientDet, and SqueezeNAS networks that we covered above all make extensive use of grouped convolutions.

Improved training regimes. A number of techniques have been developed to train a given self-attention neural network to higher accuracy on sentence classification tasks. GPT and BERT proposed methods for self-supervised pretraining of attention networks on large corpora such as Wikipedia to improve sentence classification accuracy [8, 9]. STILTS showed that, after performing the BERT pretraining scheme, accuracy can be further improved by applying transfer learning across multiple sentence classification datasets [41]. Further, MT-DNN showed that training BERT to simultaneously perform multiple sentence classification tasks can yield higher accuracy on some tasks [97].[16] In addition, RoBERTa showed that pretraining BERT for more iterations on a larger dataset yields higher accuracy on NLU tasks [13]. Further, ELECTRA trained BERT using an adversarial generator-discriminator method, and it achieved superior accuracy to RoBERTa without changing the design of the BERT encoder [12]. The RoBERTa and ELECTRA training regimes yield significant accuracy improvements when applied to both BERT-base and BERT-large, which suggests that these regimes improve the accuracy of larger (higher latency) as well as smaller (lower latency) network architectures. Finally, we believe it may be possible to further improve the accuracy of SqueezeBERT by pretraining on more data and for more iterations (similar to RoBERTa) and by pretraining with an adversarial method such as ELECTRA.

[13] FPN and EfficientDet-D1 were both pretrained on COCO and finetuned on ImageNet.

[14] FCN-8s was pretrained on ImageNet and finetuned on Cityscapes. SqueezeNAS-MAC-Small was pretrained on ImageNet and COCO and finetuned on Cityscapes. Chen *et al.* found that pretraining on COCO improved Cityscapes accuracy by approximately 2 percentage-points [93].

[15] The special case of grouped convolution with $G = C$ is known as a depthwise convolution.

[16] When running sentence classification tasks (e.g sentiment analysis *and* linguistic correctness checking), the MT-DNN [97] approach amoritizes much of the neural network computation across multiple tasks. We are interested to explore this angle for potential further speedups in future work.